

Paradigmes de Programmation
M1/GL/S1/2025/2026
Série N° 05 (Paradigme Logique)
(Durée:1H30.)

Utilisation de Swi-Prolog.

- Choisissez votre éditeur préféré pour créer un fichier avec une extension `.pl` dans lequel vous écrirez votre programme (par exemple `tp1.pl`). Attention à ne pas utiliser de majuscule pour la première lettre du nom du fichier.
- Placez-vous dans le répertoire où figure votre fichier puis lancez Prolog grâce à la commande `swipl` ou, sur certaines configurations `prolog`.
- Chargez votre programme grâce à l'instruction `[tp1]`. À chaque modification du programme, n'oubliez pas d'enregistrer et de recharger le fichier dans Prolog.

Exercice 1 (Traiter a, b, l, n et p ; le reste en homework)

On considère un arbre généalogique décrit par la base de faits suivante :

`% homme/1`

```
homme(ahmed).    % Ahmed est un homme
homme(mohamed).  homme(karim). homme(salim).
homme(ibrahim).  homme(yacine). homme(amine).
```

`% femme/1`

```
femme(fatima).   % Fatima est une femme
femme(khadija).  femme(samia). femme(safa). femme(lamia).
femme(meriem).   femme(nadia). femme(hind).
```

`% pere/2`

```
pere(ibrahim,ahmed). % Ibrahim est le père de Ahmed
pere(ahmed,mohamed). pere(mohamed,meriem). pere(ahmed,karim).
pere(yacine,fatima). pere(yacine,salim). pere(salim,nadia).
pere(salim,lamia).
```

`% mere/2`

```
mere(fatima,ahmed). % Fatima est la mère de Ahmed
mere(samia,fatima). mere(samia,salim).
mere(khadija,mohamed). mere(khadija,karim).
mere(hind,nadia). mere(hind,lamia).
mere(nadia,amine).
```

1. Définir les prédicats suivants en Prolog

- (a) **parent**(X,Y) : X est un des parents de Y
- (b) **fil**s(X,Y) : X est le fils de Y
- (c) **fil**le(X,Y) : X est la fille de Y
- (d) **femme**(X,Y) : X est la femme de Y
- (e) **mari**(X,Y) : X est le mari de Y
- (f) **gdPere**(X,Y) : X est le grand-père de Y

- (g) **gdMere**(X, Y) : X est la grand-mère de Y
- (h) **gdParent**(X, Y) : X est un des grands-parents de Y
- (i) **aGdPere**(X, Y) : X est l'arrière-grand-père de Y
- (j) **aGdMere**(X, Y) : X est l'arrière-grand-mère de Y
- (k) **frereSoeur**(X, Y) : X est le frère ou la sœur de Y
- (l) **frere**(X, Y) : X est le frère de Y
- (m) **soeur**(X, Y) : X est la sœur de Y
- (n) **beauFrere**(X, Y) : X est le beau-frère de Y
- (o) **belleSoeur**(X, Y) : X est la belle-sœur de Y
- (p) **ancetre**(X, Y) : X est l'ancêtre de Y

Exercice 2 (Traiter a, c, e et f ; le reste en homework)

On considère une base de faits décrivant les vols de la compagnie Air-Algérie:

vol(NuméroVol, VilleDépart, VilleArrivée, HeureDépart, HeureArrivée, NombrePassagers).

1. Donner plusieurs faits vol/6 selon le format ci-dessus.
2. Écrire les requêtes Prolog permettant d'obtenir la liste des vols (identifiés par leur numéro) :
 - (a) au départ d'une ville donnée ;
 - (b) à destination d'une ville donnée ;
 - (c) dont le départ aura lieu avant midi, pour une ville donnée ;
 - (d) dont l'arrivée aura lieu après 14 h, pour une ville donnée ;
 - (e) ayant plus de 100 passagers et arrivant dans une ville donnée avant 17 h ;
 - (f) partant à la même heure depuis deux villes différentes ;
 - (g) dont la durée du vol est supérieure à deux heures.

Exercice 3

On considère maintenant un réseau routier décrit par les faits suivants :

route(s, a). % une route relie la ville s à la ville a
 route(s, d).
 route(a, b).
 route(a, d).
 route(b, c).
 route(b, e).
 route(d, e).

- (a) Dessiner le graphe du réseau routier .
- (b) Exprimer les requêtes suivantes
 - quelles routes partent d'une ville donnée ?
 - quelles villes sont accessibles depuis une ville donnée?
- (c) Définir le prédicat *voisines*(X, Y) % *il existe une route directe entre X et Y*
- (d) Tracer l'arbre de résolution pour la requête : *?- voisins(d, X).*
- (e) Définir le prédicat *chemin*(X, Y) : Il existe un chemin (direct ou indirect) entre X et Y .
- (f) Quelle requête Prolog permet de trouver tous les chemins de la ville *s* vers la ville *e* ?

Exercice 4

Trois personnes, exerçant des professions différentes (médecin, ingénieur et professeur) et pratiquant des loisirs différents (peinture, jardinage et guitare), habitent dans trois maisons de couleurs distinctes (blanc, bleu, vert).

Ces trois maisons sont situées dans la même rue :

- l'une est située au début de la rue,
- une autre au milieu,
- et la troisième à la fin de la rue.

Chacune des trois maisons est donc caractérisée par un quadruplet (E,C,P,L), où :

- E est l'emplacement dans la rue,
- C la couleur de la maison,
- P la profession de l'occupant,
- L le loisir qu'il pratique.

On dispose des 5 indices suivants :

1. Dans la maison verte, on pratique la jardinage.
2. La maison verte est située avant celle où habite le professeur.
3. Le médecin habite la maison blanche.
4. La maison blanche est située avant celle où l'on pratique la peinture.
5. L'occupant qui joue de la guitare habite au début de la rue.

Déterminer les caractéristiques de chacune des trois maisons.

Paradigmes de Programmation
M1/GL/S1/2025/2026
Série N° 06 (Paradigme Logique)
(Durée: 1H30.)

Exercice 1 (Traiter 1 et 2 : a, b; le reste en homework)

Traduire en faits Prolog les tables de vérité des opérateurs logiques de base :

- la conjonction : et/3 ($a \cdot b$),
 - la disjonction : ou/3 ($a + b$),
 - la négation : non/2 ($\neg a$).
1. Définir, à partir des opérateurs de base et/3, ou/3 et non/2, les opérateurs suivants :
 - xor/3 ($a \oplus b$),
 - nor/3 ($\neg(a + b)$),
 - nand/3 ($\neg(a \cdot b)$),
 - imply/3 ($a \Rightarrow b$),
 - equiv/3 ($a \Leftrightarrow b$).

2. Vérifier les principales propriétés des opérateurs logiques :

(a) Commutativité

- $X \cdot Y = Y \cdot X$
- $X + Y = Y + X$

(b) Associativité

- $X \cdot (Y \cdot Z) = (X \cdot Y) \cdot Z$
- $X + (Y + Z) = (X + Y) + Z$

(c) Distributivité

- $X \cdot (Y + Z) = (X \cdot Y) + (X \cdot Z)$
- $X + (Y \cdot Z) = (X + Y) \cdot (X + Z)$

(d) Idempotence

- $X + X = X$
- $X \cdot X = X$

(e) Complémentarité

- $X + \neg X = 1$
- $X \cdot \neg X = 0$

(f) Théorèmes de De Morgan

- $\neg(X \cdot Y) = \neg X + \neg Y$
- $\neg(X + Y) = \neg X \cdot \neg Y$

Exercice 2 (Traiter 1, 2, 3, 7, 11 et 12 ; le reste en homework)

On souhaite traduire en Prolog les axiomes définissant les nombres naturels N et les principales opérations sur ces nombres.

Les éléments de base de la théorie des nombres naturels comprennent :

- une constante z représentant le nombre 0,
- une fonction unaire s(X) traduisant le successeur d'un nombre X,
- un prédicat unaire entier(X) signifiant que X est un nombre naturel.

1. Génération des nombres naturels

- $z \in N$
- $\forall x \in N. s(x) \in N$

2. Addition

- $\forall x \in N. x + z = x$
- $\forall x, y \in N. x + s(y) = s(x + y)$

3. Multiplication

- $\forall x \in \mathbb{N}. x \cdot z = z$
- $\forall x, y \in \mathbb{N}. x \cdot s(y) = x \cdot y + x$

4. Exponentiation

- $\forall x \in \mathbb{N}. x^z = s(z)$
- $\forall x, y \in \mathbb{N}. x^{s(y)} = x^y \cdot x$

5. Prédécesseur

- $\forall x \in \mathbb{N}. p(s(x)) = x$

6. Soustraction

- $\forall x \in \mathbb{N}. x - z = x$
- $\forall x, y \in \mathbb{N}. s(x) - s(y) = x - y$

7. Inférieur ou égal

- $\forall x \in \mathbb{N}. x \leq z \iff x = z$
- $\forall x, y \in \mathbb{N}. x \leq y \iff x = y \text{ ou } x \leq p(y)$

8. Strictement inférieur

- $\forall x, y \in \mathbb{N}. x < y \iff x \leq y \text{ et } x \neq y$

9. Quotient et reste

- $\forall x \in \mathbb{N}, \forall y \in \mathbb{N}^*. x < y \Rightarrow \text{quot}(x, y) = 0$
- $\forall x \in \mathbb{N}, \forall y \in \mathbb{N}^*. \text{quot}(x + y, y) = s(\text{quot}(x, y))$
- $\forall x \in \mathbb{N}, \forall y \in \mathbb{N}^*. x < y \Rightarrow \text{rest}(x, y) = x$
- $\forall x \in \mathbb{N}, \forall y \in \mathbb{N}^*. \text{rest}(x + y, y) = \text{rest}(x, y)$

10. Division

- $\forall x \in \mathbb{N}, \forall y \in \mathbb{N}^*. x \div y \iff \exists q \in \mathbb{N}^* \text{ tel que } x = y \cdot q$

11. Plus grand commun diviseur (pgcd)

- $\forall x \in \mathbb{N}. \text{pgcd}(x, z) = x$
- $\forall x \in \mathbb{N}, \forall y \in \mathbb{N}^*. \text{pgcd}(x, y) = \text{pgcd}(y, \text{rest}(x, y))$

12. Factorielle

- $z! = s(z)$
- $\forall x \in \mathbb{N}. s(x)! = s(x) \cdot x!$

Exercice

1. Définir un prédicat *longueur/2* tel que *longueur(L, N)* est vrai si N est le nombre d'éléments de la liste L.

Exemples :

```
?- longueur([a,b,c], N).  
N = 3.  
?- longueur([], N).  
N = 0.
```

2. Définir un prédicat *appartient/2* tel que *appartient(E, L)* est vrai si l'élément E appartient à la liste L.

Exemples :

```
?- appartient(b, [a,b,c]).  
true.  
?- appartient(x, [a,b,c]).  
false.
```

3. Définir un prédicat *concat* pour la concaténation de deux listes
4. Définir un prédicat *inverser* pour inverser une liste